Divide and Conquer

Questions and Answers

Question 1

Problem Statement

Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s. Write a program using Divide and Conquer to Count the number of zeroes in the given array.

Input Format: First Line Contains Integer m - Size of array. Next m lines Contains m numbers - Elements of an array.

Output Format: First Line Contains Integer - Number of zeroes present in the given array.

Answer (C program)

```
#include <stdio.h>

int countZero(int a[], int low, int high)
{
    if(low > high)
        return 0;

    if(a[low] == 0)
        return high - low + 1;

    int mid = (low + high) / 2;

    if(a[mid] == 1)
        return countZero(a, mid + 1, high);
    else
        return countZero(a, low, mid);
}

int main()
{
    int n;
    scanf("%d", &n);

    int a[n];

    for(int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("%d", countZero(a, 0, n - 1));

    return 0;
}
```

Output of the program

Input	Output
5 1 1 1 0 0	2

Divide and Conquer Approach

1. Look at the middle element a[mid].
2. If a[mid] is 1, all zeroes must be on the right side, so search (mid + 1 .. high).
3. If a[mid] is 0, the first zero is at mid or on the left side, so search (low .. mid).
4. If a[low] is already 0, every element from low to high is 0, so return high - low + 1.

Time Complexity

Time complexity: O(log n)
Each call discards half of the array, like binary search (a simple linear scan would be O(n)).

Question 2

Problem Statement

Given an array nums of size n, return the majority element.

The majority element is the element that appears more than floor(n / 2) times. You may assume that the majority element always exists in the array.

Example 1: Input: nums = [3,2,3] Output: 3

Example 2: Input: nums = [2,2,1,1,1,2,2] Output: 2

Constraints: n == nums.length $1 <= n <= 5 * 10^4$ $-2^{31} <= nums[i] <= 2^{31} - 1$

Example

Input	Output
3 3 2 3	3
7 2 2 1 1 1 2 2	2

Answer (C program)

```
#include <stdio.h>

int main()
{
    int n;
    scanf("%d", &n);

    int a[n];

    for(int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    int candidate = a[0];
    int count = 1;

    for(int i = 1; i < n; i++)
    {
        if(a[i] == candidate)
            count++;
        else
            count--;

        if(count == 0)
        {
            candidate = a[i];
            count = 1;
        }
    }

    printf("%d", candidate);

    return 0;
}
```

Output of the program

Input	Output
7 2 2 1 1 1 2 2	2

Divide and Conquer Approach

1. Keep a candidate and a count. Start with candidate = a[0], count = 1.
2. For every next element: same as candidate - count++, different - count--.
3. When count becomes 0, pick the current element as the new candidate with count = 1.
4. Because the majority element appears more than half the time, it is the candidate left at the end.

Time Complexity

Time complexity: O(n)
One pass over the array with O(1) extra space (Boyer-Moore voting). This answer is not a divide and conquer solution; a divide and conquer version splits the array in two, finds the majority of each half and picks the winner, giving T(n) = 2T(n/2) + O(n) = O(n log n).

Question 3

Problem Statement

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format: First Line Contains Integer n - Size of array. Next n lines Contains n numbers - Elements of an array. Last Line Contains Integer x - Value for x.

Output Format: First Line Contains Integer - Floor value for x.

Answer (C program)

```
#include <stdio.h>

int floorValue(int a[], int low, int high, int x)
{
    if(low > high)
        return -1;

    int mid = (low + high) / 2;

    if(a[mid] == x)
        return a[mid];

    if(a[mid] < x)
    {
        int result = floorValue(a, mid + 1, high, x);

        if(result == -1)
            return a[mid];

        return result;
    }

    return floorValue(a, low, mid - 1, x);
}

int main()
{
    int n, x;
    scanf("%d", &n);

    int a[n];

    for(int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    scanf("%d", &x);

    printf("%d", floorValue(a, 0, n - 1, x));

    return 0;
}
```

Output of the program

Input	Output
5 1 2 8 10 12 7	2

Divide and Conquer Approach

1. Find the middle element a[mid].
2. If a[mid] == x, then x itself is the floor.
3. If a[mid] < x, a[mid] is a possible floor: search the right half. If the right half has no floor (-1), a[mid] is the answer.
4. If a[mid] > x, the floor lies in the left half: search (low .. mid - 1).
5. If the range becomes empty, return -1.

Time Complexity

Time complexity: O(log n)
Only one half is searched in each recursive call, like binary search. The program prints -1 when no element is less than or equal to x.

Question 4

Problem Statement

Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as "No".

Note: Write a Divide and Conquer Solution

Input Format: First Line Contains Integer n - Size of array. Next n lines Contains n numbers - Elements of an array. Last Line Contains Integer x - Sum Value.

Output Format: First Line Contains Integer - Element1. Second Line Contains Integer - Element2 (Element 1 and Element 2 together sum to value "x").

Answer (C program)

```
#include <stdio.h>

int findPair(int a[], int left, int right, int x, int *p, int *q) {
    if (left >= right)
        return 0;

    int sum = a[left] + a[right];

    if (sum == x) {
        *p = a[left];
        *q = a[right];
        return 1;
    }

    if (sum < x)
        return findPair(a, left + 1, right, x, p, q);

    return findPair(a, left, right - 1, x, p, q);
}

int main() {
    int n, x, p, q;

    scanf("%d", &n);

    int a[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    scanf("%d", &x);

    if (findPair(a, 0, n - 1, x, &p, &q)) {
        printf("%d\n%d", p, q);
    } else {
        printf("No");
    }

    return 0;
}
```

Output of the program

Input	Output
5 1 3 5 7 9 12	3 9

Divide and Conquer Approach

1. Take the smallest element a[left] and the largest element a[right] and find their sum.
2. If the sum equals x, print the two numbers.
3. If the sum is less than x, move left forward: recurse on (left + 1, right).
4. If the sum is greater than x, move right backward: recurse on (left, right - 1).
5. If left >= right, no pair exists: print No.

Time Complexity

Time complexity: O(n)
Each recursive call removes one element from the range, so at most n - 1 calls are made. Recursion depth is also O(n).

Question 5

Problem Statement

Write a Program to Implement the Quick Sort Algorithm.

Input Format: The first line contains the no of elements in the list - n. The next n lines contain the elements.

Output: Sorted list of elements

Example

Input	Output
5 67 34 12 98 78	12 34 67 78 98

Answer (C program)

```
#include <stdio.h>

void quickSort(int a[], int low, int high)
{
    if(low < high)
    {
        int i = low;
        int j = high;
        int pivot = a[low];

        while(i < j)
        {
            while(a[i] <= pivot && i < high)
                i++;

            while(a[j] > pivot)
                j--;

            if(i < j)
            {
                int temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }
        }

        a[low] = a[j];
        a[j] = pivot;

        quickSort(a, low, j - 1);
        quickSort(a, j + 1, high);
    }
}

int main()
{
    int n;
    scanf("%d", &n);

    int a[n];
    for(int i = 0; i < n; i++)
```

```
        scanf("%d", &a[i]);

    quickSort(a, 0, n - 1);

    for(int i = 0; i < n; i++)
        printf("%d ", a[i]);

    return 0;
}
```

Output of the program

Input	Output
5 67 34 12 98 78	12 34 67 78 98

Divide and Conquer Approach

1. Divide: choose a[low] as the pivot and partition the array so that smaller elements are on its left and larger elements on its right (the pivot lands at index j).
2. Conquer: recursively sort the left part (low .. j - 1) and the right part (j + 1 .. high).
3. Combine: nothing to do, the array is sorted in place.

Time Complexity

Time complexity: Best / average case: O(n log n), worst case: O(n^2)
Partitioning takes O(n) per level. Balanced splits give log n levels. If the array is already sorted, the first-element pivot gives the worst case.